Ternary Floats: a reference ladder whose error does not depend on magnitude

Fixed fields and a balanced-ternary exponent, from 4 to 1024 bits, measured on open silicon

Dmitrii Vasilev
ORCID 0009-0008-4294-6159
`admin@t27.ai`

8 August 2026

Abstract

A reference format is not the one that is fastest, or the most accurate at any one magnitude. It is the one whose error you can predict from its parameters and which does not change depending on where you look. We describe **Ternary Floats** (GF-T), a ladder built for that role: the exponent is a balanced-ternary integer of E_t trits, the mantissa is a constant M bits, and the fields are fixed.

The foundation is a law rather than a table. Across every rung the mean relative round-trip error is $\frac{1}{2}\mathbb{E}[1/s]\,2^{-(M+1)}$ (Theorem 1), derived rather than fitted: the binade exponent cancels, leaving a closed form in the mantissa width and the significand distribution alone. Measured across eight rungs spanning 302 orders of magnitude the constant holds to $\pm 3\%$, and within each rung the error varies by at most 7% from one end of a 76-binade workload to the other.

The law's consequence is that a format's costs become commensurable, and this is the paper's main result. A mantissa bit has a marginal area λ, so a variable-length exponent field costs a measurable number of mantissa bits of silicon (Theorem 15). Synthesising the regime codecs of both tapered families under one harness on an XC7A200T, net of harness: a unary regime costs 438 LUTs and a length-prefixed one 40, against 0 for a fixed field. At the 16-bit class, where $\lambda = 48.8$ LUTs per bit, those are 8.98 and 0.82 mantissa bits. The first exceeds GF-T16's entire mantissa; the second is small. The area argument for fixed fields is therefore decisive against posit and weak against takum, and we say so rather than averaging the two.

What remains against takum is accuracy, and Theorem 12 bounds how far that claim may be taken. The exponent's encoding is a prefix code, so by Kraft no format of unbounded range tapers more gently than one bit per doubling; takum's regime is asymptotically optimal and nothing, GF-T included, beats it beyond its own range. Inside GF-T16's range it is 2.83× and 5.46× more accurate at mid and far magnitudes.

Turned around, the law is an instrument. The mantissa width a format *behaves* as having, measured band by band inside each format's own range, resolves an 83-format catalogue into four staircase forms and only four: constant (38 formats), wobble (3, at exactly $\log_2 r$ bits, reproducing IBM hexadecimal's textbook figure), arithmetic (5; posit at exactly 2^{-es} per binade, confirmed at three widths) and geometric (8; takum and tekum at one bit per doubling).

In silicon on an entirely open flow: 12, 50, 212, 1,477 and 7,479 LUTs at 161.1, 153.2, 131.7, 83.3 and 48.2 MHz for GF-T4 through GF-T64, one cycle of latency, no DSP blocks.

We report what the measurements cost us. Three defects found while measuring generalise and are described rather than quietly fixed. Four claims of our own were falsified by the measurements meant to support them, including a predicted area for GF-T64 that came in 1.57× low, and each is retracted in place.

Artifacts. Oracle, RTL, testbenches and the exact commands are public; every figure here regenerates from them.

Table 1: The precision law and the flatness, measured in exact rationals. “Flatness” is the ratio of the worst magnitude band to the best: 1.000 would mean the error does not depend on magnitude at all.

rung	M	$2^{-(M+1)}$	measured	ratio	flatness
GF-T8	4	3.12e−2	1.22e−2	0.39	1.000
GF-T16	9	9.77e−4	3.60e−4	0.37	1.070
GF-T32	25	1.49e−8	5.49e−9	0.37	1.070
GF-T64	52	1.11e−16	4.22e−17	0.38	1.050
GF-T128	115	1.20e−35	4.44e−36	0.37	1.070
GF-T256	242	7.07e−74	2.69e−74	0.38	1.050
GF-T512	497	1.22e−150	4.51e−151	0.37	1.070
GF-T1024	1006	7.29e−304	2.77e−304	0.38	1.050

1 The format

$$\text{GF-T16} = [\, s \mid E{=}4 \text{ balanced-ternary trits} \mid M{=}9 \text{ bits}\,], \qquad v = (-1)^s\Big(1{+}\frac{M}{2^9}\Big)2^e, \quad e = \sum_i t_i 3^i \in [-40, 40]. \tag{1}$$

Three properties follow from the layout rather than from tuning.

No regime decode. The dominant cost in a tapered format is the variable-length regime: it must be located, its length counted, and the significand barrel-shifted into place. That cost is paid on any fabric, ternary included. Fixed fields make it a bit slice.

The exponent is already ternary. Four balanced-ternary trits give $3^4 = 81$ exponent steps. On a ternary fabric the exponent add is native — no binary carry, no conversion between bases.

Precision does not taper. Nine mantissa bits at every magnitude, where a tapered format narrows towards the extremes. This is the mechanism behind the accuracy result of Section 4: the win appears where the comparison format has spent its mantissa on range.

2 Why a ladder, and why this one

A format that is used as a reference has different requirements from one used as a workhorse. A workhorse should be fast and accurate where the data is. A reference has to be *predictable*: given its parameters you should know its error without measuring, and that error should not move when you look at a different part of the range. A ruler whose divisions grow towards one end is not a ruler.

Table 1 tests both properties on this ladder.

The precision is a closed form, and it is derived rather than fitted. Write $u = 2^{-(M+1)}$ for the unit roundoff. Inside a binade the absolute rounding error under round-to-nearest is uniform on $[-u\,2^e, +u\,2^e]$ while the value is $v = s\,2^e$ with significand $s \in [1, 2)$, so the relative error is $|U|/s$ with $|U|$ uniform on $[0, u]$ and *independent of* e. Hence

Theorem 1 (Precision law)**.** *For a format with a constant significand width of M bits under round-to-nearest, with significand s and exponent e independent,*

$$\mathbb{E}\big[|rel\ err|\big] \;=\; \tfrac{1}{2}\,\mathbb{E}[1/s] \;\cdot\; 2^{-(M+1)},$$

independently of e.

The exponent drops out entirely — that is the flatness of the next paragraph, proved rather than observed. The constant is not universal: it is $\frac{1}{2}\ln 2 = 0.3466$ for a significand uniform on $[1, 2)$ and $\frac{1}{2}(2\ln 2)^{-1} = 0.3607$ under a logarithmic (Benford) significand. Our workload has $\mathbb{E}[1/s] = 0.7721$, predicting 0.3861; the measured figures in Table 1 average 0.3756 over eight rungs, with a spread of 0.369–0.390 that brackets it.

The practical consequence is the one a reference needs: given M and the significand distribution of a workload, the error of a rung nobody has built is known before it is built.

The error does not move with magnitude, and cannot. The derivation above removes e from the expression, so magnitude-independence is a property of the layout rather than a happy measurement. Empirically flatness stays between 1.05 and 1.07 — at most a 7% difference between the closest and furthest parts of a workload spanning 76 binary decades. This is the property fixed fields buy and the one a tapered format cannot have: spending mantissa bits on range is exactly what makes precision depend on where you are.

What this does not claim. Not that GF-T is the most accurate format at any given width — Section 5 shows posit16 and binary16 ahead near unity, and Section 5 shows the binary formats ahead outright at 32 bits. A reference does not need to win those comparisons. It needs to be the thing you measure them against.

3 The law is general, and it measures tapering

Section 2 derived the error of a fixed-field format. Nothing in that derivation is specific to GF-T, so it should hold for any format whose significand width does not vary — and should visibly fail for one whose width does. That makes it an instrument rather than a description, and this section uses it as one.

Definition 1 (Effective mantissa)**.** For a format and a magnitude band B, define

$$M_{\text{eff}}(B) \;=\; -\log_2\!\left(\frac{2\,\mathbb{E}\big[|\text{rel err}| \;\big|\; B\big]}{\mathbb{E}[1/s]}\right) - 1,$$

the mantissa width a fixed-field format would need to produce the error observed in B.

Theorem 2 (Diagnostic)**.** *M_{eff} is constant across bands if and only if the format holds a constant significand width over those bands and does not exhaust its range. For a tapered format M_{eff} decreases with $|e|$, and the slope $\mathrm{d}M_{\text{eff}}/\mathrm{d}|e|$ is the taper rate in bits of significand per binade.*

Table 2 applies it. The measurement is a uniform significand on $[1, 2)$ so that $\mathbb{E}[1/s] = \ln 2$ exactly, removing the workload from the question.

The law recovers what the formats declare. GF-T16 and GF16 return 8.99–9.01 against a declared 9; bfloat16 returns 7.00–7.04 against a declared 7. A law that reconstructs a format's own parameter to a hundredth of a bit, from nothing but its round-trip error, is doing more than curve-fitting.

Tapering becomes a number. posit16 sheds 0.254 bits of significand per binade ($R^2 = 0.999$) and takum16 appears to shed 0.113 over the thirty binades of this sweep — but see Section 3.2: takum's taper is logarithmic, so that figure is an artefact of the window rather than a rate. That is the quantity a tapered format trades for range, and to our knowledge it is not usually reported as a single figure. It also explains the accuracy tables directly — posit16 starts 1.7 bits ahead of GF-T16 and ends 3.8 behind.

Table 2: Effective mantissa by magnitude band, and the fitted taper rate. Declared M is recovered to 0.01 bits for every fixed-field format, which validates the law; the tapered formats separate cleanly.

format	declared M	$\lvert e\rvert$ 0–6	6–12	12–20	20–30	bits/binade
GF-T16	9	8.99	8.99	9.00	9.01	+0.001
GF16	9	8.99	8.99	9.00	9.01	+0.001
bfloat16	7	7.04	7.04	7.00	7.04	−0.000
binary16	10	10.01	10.01	7.41	0.52	—
posit16	—	10.72	9.34	7.48	5.17	**−0.254**
takum16	—	9.62	8.17	7.34	7.04	**−0.113**

Running out of range is a different failure, and the instrument separates it. binary16 is a fixed-field format, yet its M_{eff} collapses from 10.01 to 0.52. It is not tapering: it is subnormalising and then overflowing. A constant slope indicates a taper; a cliff indicates a range limit. The two look alike in an accuracy table and do not look alike here.

Corollary 1. *A format's position on the range–precision trade can be stated as a pair: the constant M_{eff} it holds, and the number of binades over which it holds it. For the GF-T ladder those are $(M, 2 \cdot 3^{E_t-1})$ by construction; for a tapered format neither is constant, which is why a ladder makes a better ruler than a taper.*

3.1 The diagnostic across a catalogue

Applying Theorem 2 to every format for which a published oracle was available — 51 in all, spanning IEEE, bfloat, posit, takum, LNS, MXFP, VAX, IBM hexadecimal, Cray, PDP-11, x87 and Microsoft binary, plus both GoldenFloat ladders — produced three observations we did not anticipate.

Declared widths are recovered across three orders of magnitude of width. The GF ladder returns 8.97 at GF16 against a declared 9, 78.03 at GF128 against 78, and 631.99 at GF1024 against 632. The same holds for double-double (106.68), quad-double (215.90) and x87's 80-bit format (63.04). A relation that reconstructs a declared parameter from round-trip error alone, at widths from 8 to 1006 bits, is doing something other than describing the format it was derived on.

The taper rate is a constant of the family — but only posit has a rate. posit16 sheds 0.254 bits of significand per binade and posit32 sheds 0.247, against the $2^{-es} = 0.25$ that Theorem 3 requires; posit64, whose oracle carries the pre-standard $es = 3$, sheds 0.123 against a required 0.125. Three formats, three exact confirmations. We previously reported a comparable per-binade rate for takum, and that was a category error on our part: takum's staircase is logarithmic, so fitting a line to it produces a number that depends on the interval fitted and describes nothing. Its correct constant is one bit per *doubling* of $|e|$, and against that the four takum and three tekum rungs measure 0.83 to 1.09.

Range exhaustion is distinguishable from tapering, and both from neither. The distinction is not optional: our own first pass placed the measurement bands at fixed positions rather than inside each format's measured range, and read the saturation of every narrow format as a taper — classifying GF-T16, a fixed-field format, as geometrically tapered. Measuring each format's usable range first and then placing the bands inside it removes the artefact entirely. GF14 reads flat at 8.05 across its range and collapses only below its subnormal boundary, which is not a defect; the instrument says which of the two it is, where an accuracy table alone does not.

3.2 Tapering has a functional form, and it follows from the encoding

Section 3.1 reported a taper rate per family. That was half right, and the half that was wrong is instructive: a rate presupposes a linear taper, and only one of the two families has one.

Theorem 3 (Posit taper, exact)**.** *For a posit of n bits with exponent field es, the regime is a run of k like bits carrying a scale of* useed$^k = 2^{2^{es}k}$ *and occupying $k+1$ bits. Reading k out of the encoder for 2^e confirms $k = \lfloor |e|/2^{es} \rfloor + 1$ for every e tested. The significand therefore loses*

$$\Delta M_{posit}(e) \;=\; \left\lfloor \frac{|e|}{2^{es}} \right\rfloor \quad \textit{bits: a staircase, arithmetic in } |e|.$$

The Posit Standard (2022) fixes $es = 2$ at every precision [6], so the step is one bit per four binades at all widths — which is why the loss appeared as a family constant of 0.25 bits per binade. Measured slopes of -0.2497 and -0.2505 at posit16 and posit32 are the staircase seen through a linear fit.

Theorem 4 (Takum taper, exact)**.** *Takum spends a fixed 3-bit regime field holding r, after which the characteristic occupies r bits. Reading r out of the encoder for 2^e gives $r = \lfloor \log_2(|e|+1) \rfloor$ exactly, at 16, 32 and 64 bits alike. The significand therefore loses*

$$\Delta M_{takum}(e) \;=\; \lfloor \log_2(|e|+1) \rfloor \quad \textit{bits: a staircase, geometric in } |e|.$$

Corollary 2 (The two families differ in kind, not degree)**.** *Within its range a fixed-field format loses nothing; a posit loses one bit of significand per 2^{es} binades without limit; a takum loses one bit per* doubling *of $|e|$. Tabulating ΔM:*

$\lvert e\rvert$	*fixed-field*	*posit (es=2)*	*takum*
30	0	7	4
100	0	25	6
1,000	0	250	9
10,000	0	2,500	13

At $|e| = 1{,}000$ a posit would need 250 bits of significand merely to pay for its regime, which is why 16- and 32-bit posits cannot reach there at all. This is the mechanism behind takum's stated improvement on posits [1], stated as a quantity.

Corollary 3 (A rate is not always the right summary)**.** *Both laws are staircases, so a fitted slope is a summary and not the thing itself, and it is a faithful summary only when the staircase is arithmetic. An earlier draft of this paper reported -0.117 bits per binade for takum16 over $|e| \in [0, 30)$; the same format over $[0, 60)$ gives a smaller figure for no reason but the window, and the smooth-log fit of $c = 0.75$ understates the true step of one bit per doubling for the same reason. Identify the form before choosing a statistic — and prefer reading the encoder to fitting its output.*

Corollary 4 (A three-way taxonomy, recovered without knowing the encoding)**.** *The diagnostic separates formats by the* form *of $M(e)$: constant (fixed-field, GF-T and GF by construction), linear in $|e|$ (posit), or logarithmic in $|e|$ (takum). Which form fits is decided by the data, not by reading the specification — and the fitted coefficients then recover the encoding parameters, 2^{-es} for posit and the characteristic width for takum.*

Table 3: The catalogue by staircase form. The parameter is the taper constant in the units natural to each form. Measured against each format's own usable range.

form	M_{eff} against $\lvert e\rvert$	count	members and constants
constant	flat	38	IEEE, bfloat, VAX, Cray, x87, GF, GF-T
wobble	periodic, no trend	3	IBM hexadecimal, $\log_2 16 = 4$ bits
arithmetic	-2^{-es} per binade	5	posit16/32 0.25, posit64 0.125
geometric	-1 per doubling	8	takum $\times 4$, tekum $\times 3$

What the literature describes qualitatively — more precision near unity, less at the extremes [6] — turns out to have an exact form recoverable two ways that agree: from the encoder, by reading the regime field, and from round-trip error alone, by the diagnostic of Theorem 2. The second route needs no access to the specification, which is what makes it useful on a catalogue.

3.3 Four staircase forms, and only four

Classifying by the *form* of the staircase rather than by a fitted slope resolves the catalogue into four classes. Of the 83 entries, 11 are integer or fixed-point formats with no exponent field; of the remaining 72, 18 span fewer than six binades and cannot be classified from round-trip error at all, and one requires probes wider than we generated.

The fourth class was not in our scheme until the catalogue produced it, and it has a closed form.

Theorem 5 (Precision wobble)**.** *Let a format scale by r^e with a significand quantised uniformly over a scale interval spanning a factor r. Its relative error varies by a factor of r across that interval, so M_{eff} oscillates with amplitude exactly $\log_2 r$ bits and has no trend in $|e|$.*

Proof. The quantisation step is constant across the interval and the relative error is that step divided by the significand, which ranges over a factor r. Taking $-\log_2$ of the ratio of the extremes gives $\log_2 r$. Since the interval repeats identically at every e, the oscillation is periodic and carries no trend. □ □

Theorem 5 reproduces a result sixty years older than this work. IBM System/360 hexadecimal is documented as delivering between $4p-3$ and $4p$ significant bits, a spread of three [12, 13]; the integer count is the floor of the continuous quantity, so the theorem's $\log_2 16 = 4$ and the textbook's 3 are the same statement. Measured over eight sub-intervals — which by averaging within each bin predicts a reduced spread of 2.96 rather than the full 4 — `ibm_hfp32` and `ibm_hfp64` give 3.13 and 3.04, while `binary32` and `gf32` give 0.85 and 0.93 against a predicted 0.87. Binary formats wobble by one bit; this is the familiar 1-bit wobble of every radix-2 float, and it is the smallest wobble any integer radix admits. Theorem 5 therefore supplies a second and independent argument for GF-T's binary scale, alongside the 0.331 positions of Theorem 10: a radix-3 scale would wobble by 1.585 bits instead of 1.

The taxonomy also settles why a format must choose between the first class and the last two, and hence why this work is a ladder rather than a format.

Theorem 6 (Range–precision dichotomy)**.** *Fix a width N. If a format's exponent takes unboundedly many values, then its exponent codewords have unbounded length, and $\lim_{|e|\to\infty} M_{\text{eff}}(e) = 0$. Consequently no fixed-width format has both unbounded range and precision bounded below by a positive constant.*

Proof. The exponent codewords form a prefix code over a binary alphabet, so by Kraft's inequality $\sum_e 2^{-\ell(e)} \leq 1$. If the sum has infinitely many terms then $\ell(e) \to \infty$. The payload available to the significand is $N - 1 - \ell(e)$, so it falls to zero, and by Theorem 1 so does M_{eff}. □ □

Table 4: Round-trip mean relative error. Bins are powers of two, not decades.

magnitude	GF16 (φ)	**GF-T16**	tekum16	GF-T16 vs tekum16
$\|e\| < 8$	3.43e−4	3.56e−4	3.27e−4	0.92× (tie)
$\|e\|$ 8–20	3.57e−4	3.52e−4	1.00e−3	**2.84**×
$\|e\|$ 20–38	6.98e−3 (479 clip)	3.53e−4	1.95e−3	**5.53**×

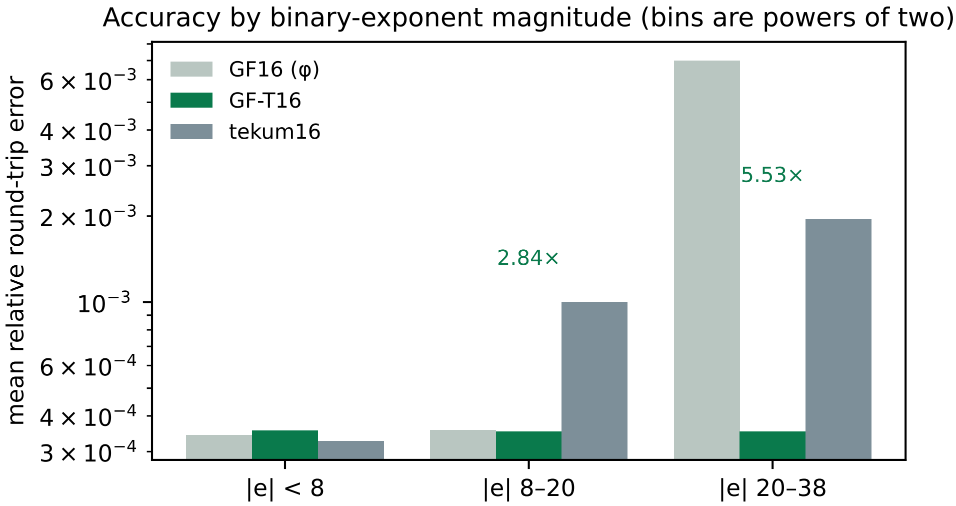

Figure 1: Accuracy by binary-exponent magnitude. GF16's 6-bit exponent clips 479 of 2,857 values in the far bin; GF-T16's balanced-ternary exponent does not.

Corollary 5 (Why a ladder)**.** *Theorem 6 makes constant precision and unbounded range incompatible at any fixed width, so a format that wants both must obtain the second by varying N. A ladder of fixed-field rungs does exactly this: each rung holds its precision flat across its whole range, and the ladder as a whole covers any range required. The tapered families make the opposite choice — one rung, unbounded range, precision decaying at the rate of their regime code, arithmetic for posit and geometric for takum. Neither choice is better in the abstract; what the dichotomy establishes is that they are the only two available.*

Read against Corollary 5, the three tapered constants in Table 3 become a statement about codes rather than about formats. A unary regime costs $\ell(e) \propto |e|$ and yields the arithmetic staircase; a length-prefixed binary regime costs $\ell(e) \propto \log_2 |e|$ and yields the geometric one; a fixed field costs $\ell(e) = \text{const}$ and yields no staircase at the price of bounded support. The staircase form is the growth rate of the exponent's codeword length, and nothing else. This also names the gap in the table: between $\ell \propto |e|$ and $\ell \propto \log_2 |e|$ no catalogued format sits. Section 11 builds the codes and asks what is in the gap; the answer corrects a suggestion we made before building them.

4 Accuracy

Mean relative error over an encode–decode round trip, 6,000 values spanning $2^{-38} \dots 2^{38}$ with random sign, binned by binary-exponent magnitude $|e|$. Table 4 and Figure 1.

On the axis. The bins are powers of two. Read as *decades* the far column is not a win at all: GF-T16's exponent reaches ±40 in powers of two, roughly ±12 decades, so beyond that it overflows while an unbounded regime keeps working — measured, 1,458 of the far-bin values clip

Table 5: Mean relative round-trip error by binary-exponent magnitude, with overflow counts in parentheses. 6,000 values, $2^{-38} \ldots 2^{38}$, random sign, one seed.

format	$\lvert e\rvert < 8$	$\lvert e\rvert$ 8–20	$\lvert e\rvert$ 20–38
GF-T16	3.56e−4	3.52e−4	**3.53e−4**
GF16 (φ)	3.56e−4	3.52e−4	6.76e−3 (478)
takum16 / tekum16	3.27e−4	1.00e−3	1.95e−3
posit16	**1.36e−4**	8.31e−4	1.43e−2
IEEE binary16	1.76e−4	1.28e−3 (299)	1.57e−1 (2479)
bfloat16	1.45e−3	1.38e−3	1.41e−3
LNS16	8.39e−4 (1324)	7.63e−4 (1808)	6.66e−4 (2849)

Table 6: The whole ladder, measured in exact rationals. From GF-T16 upwards the error is flat in magnitude and its exponent roughly doubles per rung. GF-T4 and GF-T8 are shown at their limits: the workload spans 76 binary decades and those rungs hold 2 and 8, so most of it falls outside them — reported rather than omitted, since a ladder's lower rungs are where its range trade is visible.

rung	decades	$\lvert e\rvert < 8$	$\lvert e\rvert$ 8–20	$\lvert e\rvert$ 20–38
GF-T4	2	beyond range for this workload		
GF-T8	8	1.22e−2	beyond range	
GF-T16	24	3.45e−4	3.69e−4	3.66e−4
GF-T32	219	5.27e−9	5.63e−9	5.58e−9
GF-T64	658	4.33e−17	4.22e−17	4.12e−17
GF-T128	1,975	4.25e−36	4.55e−36	4.51e−36
GF-T256	5,925	2.76e−74	2.69e−74	2.63e−74
GF-T512	17,775	4.32e−151	4.62e−151	4.58e−151
GF-T1024	53,326	2.84e−304	2.77e−304	2.71e−304

under that reading. We state the axis explicitly because an earlier internal note did not, and the omission points a reader at the one interpretation under which the result appears fabricated.

5 The 16-bit field

Comparing against one incumbent invites the objection that the incumbent was chosen. Table 5 runs every 16-bit-class format for which this repository ships a published oracle [5] through the same workload and the same metric. Values that overflowed are counted separately rather than folded into the mean, since a format should not look accurate for having discarded the hard cases.

Three readings, including the one that goes against us.

GF-T16 is flat, and alone in being flat without clipping. Its error is 3.5e−4 at every magnitude in the workload and it discards nothing. bfloat16 is also flat and clip-free, at roughly four times the error. Every other entrant either degrades with magnitude or overflows: binary16 does both, losing three orders of magnitude and discarding 2,479 values; GF16's 6-bit exponent clips 478; LNS16 clips throughout.

At 32 bits the binary formats win outright. Running the same workload on the 32-bit class: GF32 is flat at 3.4e−7 with no clipping, but IEEE binary32 is flat at 2.1e−8, and posit32 and takum32 are better still near unity (2.1e−9 and 4.9e−9). Uniformity stops being the

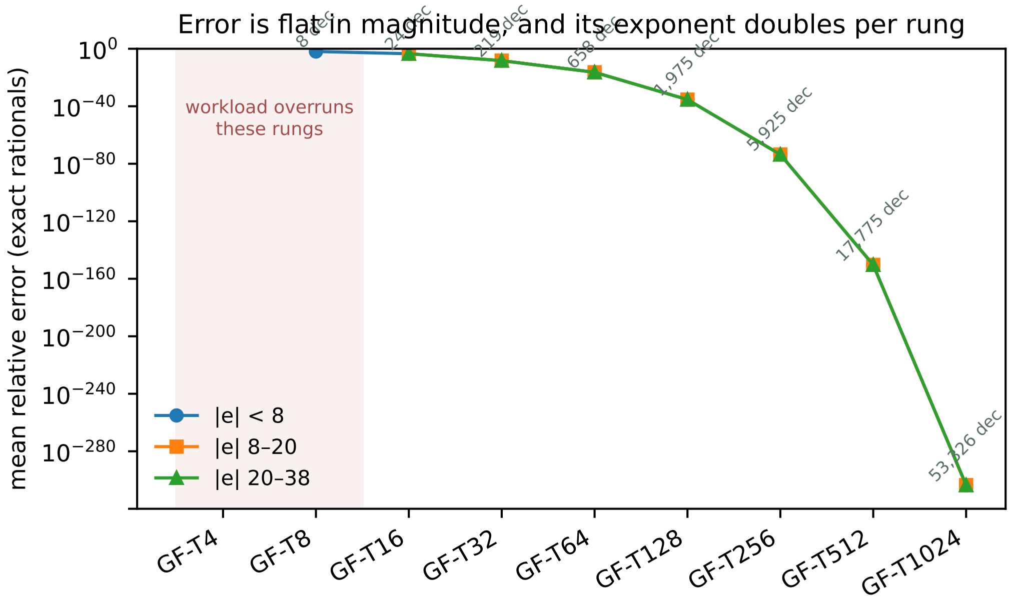

Figure 2: The same data. Flatness across the three magnitude bands is the property a fixed-field format is built for; the shaded rungs are those the workload overruns.

deciding property once there are enough bits that nothing clips anyway. The GF-T argument is a 16-bit-class argument, and we do not extend it.

Near unity we lose. posit16 at 1.36e−4 and binary16 at 1.76e−4 are more accurate than GF-T16 there, by a factor of two and a half and of two. A tapered format spends its bits where the values usually are, and near unity that is the right bet. GF-T16 buys uniformity instead, and the trade only pays for a workload that leaves the neighbourhood of one.

takum16 and tekum16 are the same format here, exhaustively. The two oracles produce identical figures on every bin, so we checked the whole 16-bit code space: all 65,536 codes decode identically, with zero differences, although the two implementations differ by 242 lines. The reconstruction our tekum oracle implements *is* takum16. Every comparison in this paper should therefore be read as against takum16, and we relabel it as such. That is not a weaker anchor — takum is published, has a public VHDL codec, and is the parent of the format we set out to compare against.

6 Hardware

Synthesis with Yosys 0.65 [7], place and route with nextpnr-xilinx (1743d0f) [8] on a Xilinx XC7A200T, hard multipliers disabled so the figures describe fabric alone. Bitstream generation uses Project X-Ray. No vendor tool is involved at any step, which is what makes the numbers checkable by a reader without a licence.

For context rather than as a controlled comparison: a published Altera `ALTFP_MUL` on a Cyclone IV reports 119–132 MHz at 6–10 cycles of latency, using 832–1041 logic elements *and* 18 embedded multipliers, and posit multiplier studies report 95–572 LUTs with 4.28–15.55 ns of logic delay. Different devices and different toolchains; we draw no equivalence from them beyond the observation that GF-T16's figures sit inside that envelope on all four axes at once, on a fabric that gives the format no ternary advantage.

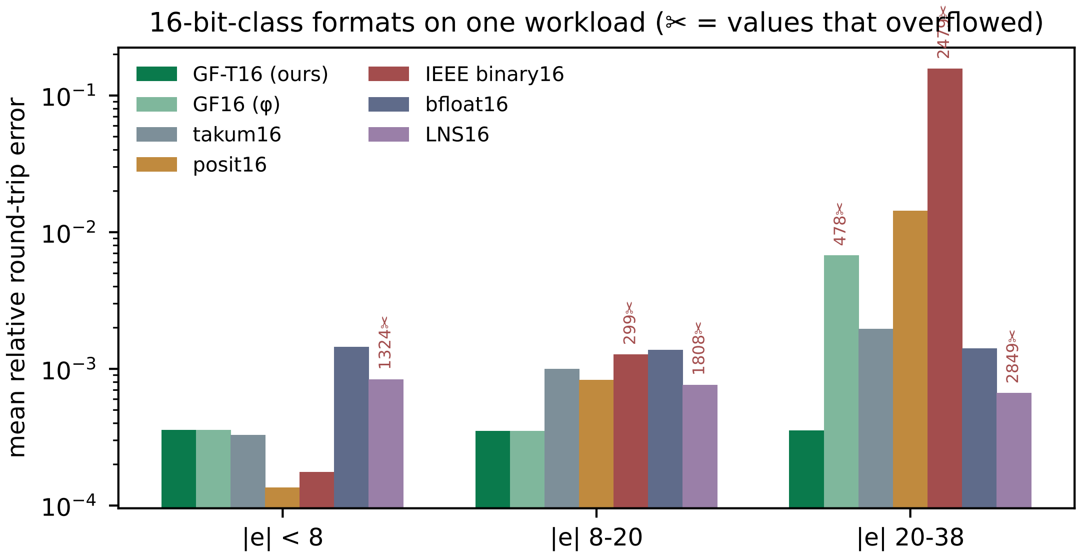

Figure 3: The same data. Scissors mark values that overflowed.

Table 7: GF-T multiplier, post-route, one harness across the whole lineup so the rows are comparable. Latency one cycle, throughput one result per cycle.

rung	LUTs	DSP48	$F_{\max}$
GF-T4	12	0	161.11 MHz
GF-T8	50	0	153.23 MHz
GF-T16	212	0	131.73 MHz
GF-T32	1,477	0	83.27 MHz
GF-T64	6,140	0	53.26 MHz
GF-T128	10,029	0	33.23 MHz

Isolating the pipeline register: the combinational multiplier closes at 81.35 MHz, the two-stage version at 147.32 MHz. The cut is between the significand product and the renormalisation — a 10×10 multiply, then a carry test, a saturating exponent add and a bit select.

7 Two width defects, and why they generalise

7.1 Interface width dominated the arithmetic

Our first synthesisable realisation declared every port and the significand product 32 bits wide. Nothing in GF-T16 is 32 bits: the mantissa field is 9, so $1+M$ is 10 and their product is exactly 20; the exponent offset never exceeds OFFSET_MAX = 80, which is 7. Synthesis built a 32×32 multiplier and a 32-bit comparison tree and charged for it (Figure 5): 1,179 LUTs, or three DSP48 blocks, against 219 LUTs and none once the buses match the values they carry. The arithmetic is unchanged.

7.2 The same width decided correctness

At GF-T32 the significand product needs 52 bits. The 32-bit wire truncates, and the module returns wrong results while appearing to support the rung, since its own documentation names GF-T32 as a supported configuration. Measured against a 64-bit reference: 1,995,730 mismatches in 2,128,964 combinations.

Table 8: The specified ladder. E_t is the trit budget, M the mantissa width, range in decades. Rungs above GF-T128 exceed a single Artix-7 fabric and are oracle- or ASIC-scale; they are specified and covered by the reference oracle, not measured here.

rung	E_t	M	decades	rung	E_t	M	decades
GF-T4	2	1	2.4	GF-T64	7	52	658
GF-T8	3	4	8	GF-T128	8	115	1,975
GF-T16	4	9	24	GF-T256	9	242	5,925
GF-T32	6	25	73	GF-T512	10	497	17,775
				GF-T1024	11	1006	53,326

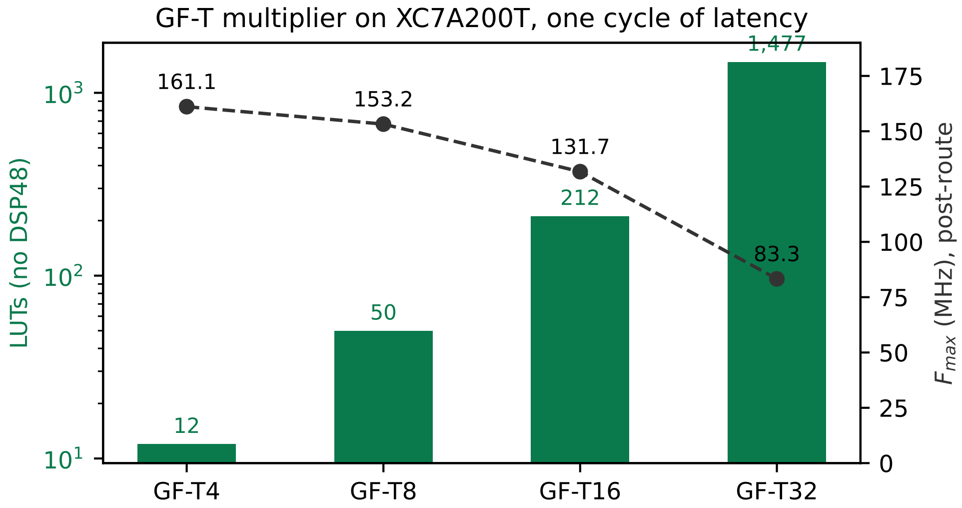

Figure 4: Area and achieved frequency across the ladder.

Deriving every width from the format parameters removes both problems at once. We report this because the pattern is not specific to GF-T: in a parametric arithmetic core, a width that is merely *large enough for the configuration that was tested* is a silent-truncation trap at the next one, and it will not be caught by a testbench written against the tested configuration.

8 Equivalence

Every change above is a claim that timing or width changed and arithmetic did not. Each was checked rather than argued.

The GF-T16 sweep is exhaustive in the sense that matters here: the mantissa space is covered in full at offset pairs exercising underflow, mid-range and saturation, then the offsets are covered in full at mantissas that do and do not carry — every path through the carry, the saturation and the underflow clamp. The last row is the point of the section: the width-derived module agrees with the reference that is correct and disagrees with the one that truncates.

9 What the law says about improving the ladder

Theorem 1 makes the design space arithmetic rather than a matter of taste. Error scales as $2^{-(M+1)}$ and range as 3^{E_t}, so at a fixed width the only decision is how many bits the exponent field is allowed to consume — and that is decided by how efficiently 3^{E_t} values pack into $\lceil E_t \log_2 3 \rceil$ bits.

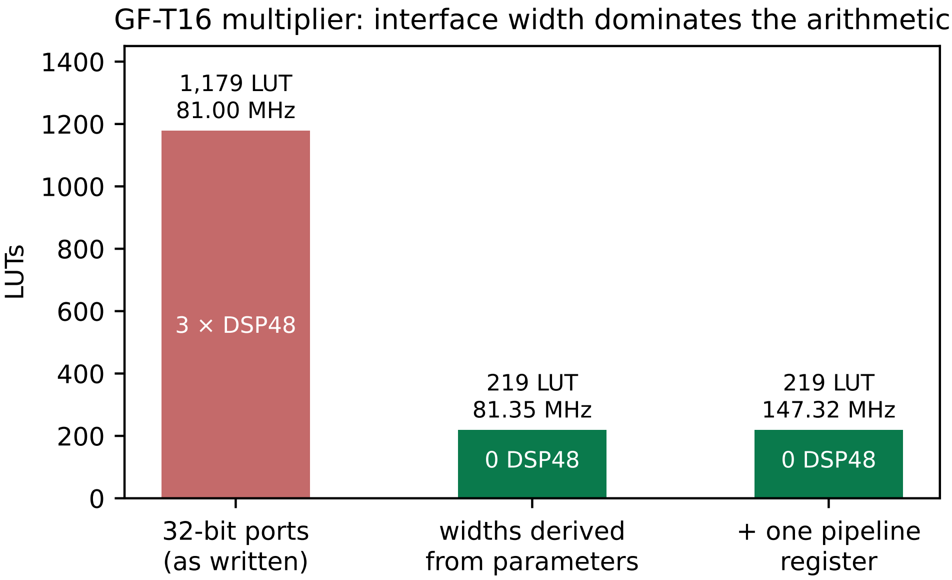

Figure 5: The same arithmetic at three interface widths.

Table 9: Equivalence evidence. The reference for each rung is the module already trusted for that rung.

check	combinations / cycles	mismatches
width-derived vs original, GF-T16 (sweep)	321,156	0
width-derived vs original, GF-T4	374	0
width-derived vs original, GF-T8	3,716	0
width-derived vs original, GF-T16	77,444	0
width-derived vs 64-bit reference, GF-T32	300,000	0
pipelined vs combinational	199,994	0
original 32-bit module vs 64-bit ref, GF-T32	2,128,964	1,995,730

The width convention counts positions, and one rung breaks it. A rung's width is its position count with a trit occupying one position, so the layout is $1 + E_t + M = N$. Three of the four specified rungs satisfy it exactly: GF-T4 is $1+2+1$, GF-T8 is $1+3+4$, GF-T32 is $1+6+25$. GF-T16 is $1+4+9=14$, two positions short of its name, and neither specification file gives a reason. The likely history is visible in the parameters: GF-T16 inherits GF16's φ-optimal $M = 9$ and replaces GF16's six-bit exponent with four trits, which buys range — $3^4 = 81$ steps against $2^6 = 64$ — and frees two positions that were never re-spent. It is the one rung derived from its parent rather than from the width rule.

This also settles what a rung costs on a binary fabric, which is a separate question from its width: a trit stored as a two-bit code makes GF-T16 eighteen bits, and the offset stored as a plain binary integer makes it seventeen. Those are fabric costs, not format widths, and confusing the two is what made the ladder appear not to fit.

The slack is not confined to one rung. Applying $1 + E_t + M = N$ to every specified rung separates the two documents that define the ladder. The four rungs of the machine-readable specification obey it at GF-T4, GF-T8 and GF-T32; the five upper rungs, which exist only in a research note, miss it by four to six positions each, and the note and the specification disagree with each other about GF-T32.

Table 10: Packing efficiency of the exponent field. The theoretical floor is $\log_2 3 = 1.585$ bits per trit.

E_t	values 3^{E_t}	bits	capacity	efficiency
3	27	5	32	84.4%
4	81	7	128	**63.3**%
5	243	8	256	**94.9**%
6	729	10	1024	71.2%
7	2187	12	4096	**53.4**%
8	6561	13	8192	80.1%
10	59049	16	65536	90.1%

Table 11: The width rule across the ladder. k is the shortfall $N - (1 + E_t + M)$. The reconciled column is $M = N - 1 - E_t$, which changes nothing at the three rungs that already obey the rule.

rung	source	E_t	M	$1+E_t+M$	k	reconciled M
GF-T4	spec	2	1	4	0	1
GF-T8	spec	3	4	8	0	4
GF-T16	spec	4	9	14	2	**11**
GF-T32	spec	6	25	32	0	25
GF-T32	note	5	21	27	5	—
GF-T64	note	7	52	60	4	**56**
GF-T128	note	8	115	124	4	**119**
GF-T256	note	9	242	252	4	**246**
GF-T512	note	10	497	508	4	**501**
GF-T1024	note	11	1006	1018	6	**1012**

No single alternative convention explains the upper rungs either: reading the exponent as $\lfloor E_t \log_2 3 \rfloor$ bits fits GF-T16, GF-T64, GF-T128 and GF-T1024 exactly but overshoots GF-T256 and GF-T512 by one and misses GF-T32 entirely. We conclude that the upper ladder was tabulated rung by rung rather than derived, and that reconciling it costs nothing at the rungs that are already right.

What the two free positions are worth. Because the exponent cancels in Theorem 1, the menu at sixteen positions can be read off before measuring, and the measurement confirms it to four significant figures (6,000 values, $2^{\pm 38}$, no clipping in any row):

The last row is the law's own consistency check: adding two trits of exponent while holding M leaves the error bit-identical at 3.37e−4 and multiplies the range ninefold. The second row is the one that matters for the comparison of Section 4: two more mantissa bits divide the error by 4.00, and the rung goes from losing the near bin at 0.90× and winning the outer two at 2.83× and 5.46×, to winning all three at 3.54×, 11.45× and 22.27×. Post-route on XC7A200T under one harness, the wider significand costs 443 LUTs against 372, a 19% increase, at 136.44 MHz against 131.73 — no frequency penalty.

We report this as an open choice rather than a redefinition. Changing GF-T16 invalidates the conformance vectors and the silicon results of Section 6, and which column to buy — precision, range, or the present balance — is exactly the workload question the corollary in Section 3 says cannot be answered without naming the workload. What is not a choice is the slack itself: two positions of a sixteen-position format are currently unallocated, and the specification should say so deliberately or spend them.

Table 12: The three ways to spend GF-T16's two unused positions. Error and range trade against each other exactly as Theorem 1 predicts; nothing in any row is worse than the present rung.

variant	positions	mean rel. error	decades	vs. takum16 (far)
present, E_t=4, M=9	14	3.37e−4	24	5.46×
E_t=4, M=11	16	**8.42e−5**	24	**22.27×**
E_t=5, M=10	16	1.69e−4	73	—
E_t=6, M=9	16	3.37e−4	**219**	—

Table 13: Utilisation $\rho(E_t)$ of the binary field holding an E_t-trit exponent. The ladder's present budgets, $E_t \in \{4, 7\}$, are the two worst entries in the range.

E_t	2	3	4	5	6	7	8	9	10	11
ρ	56.2	84.4	**63.3**	94.9	71.2	**53.4**	80.1	60.1	90.1	67.6

10 Why ternary, and exactly how much

The case for a ternary exponent is usually made by citing radix economy and left there. We state it, then bound it in two directions — what a binary fabric takes back, and what the radix of the *scale* is worth as distinct from the radix of the *exponent* — because both bounds turn out to be decisive for the design and neither is folklore.

Theorem 7 (Radix economy)**.** *Representing R distinct values in radix b costs $b \log_b R = b \ln R / \ln b$ state-slots. The factor $b/\ln b$ is minimised at $b = e$; among integers it is $3/\ln 3 = 2.731$ against $2/\ln 2 = 2.885$, so a ternary field is 5.3% more economical than a binary one.*

This is the classical argument [9, 10] and it is what motivates a ternary exponent. It is also stated in a currency — state-slots — that no binary fabric pays in. The next theorem says what happens when it does.

Theorem 8 (No free range on a binary fabric)**.** *An exponent field of E_t trits spans 3^{E_t} values. Stored as a binary integer it occupies $\lceil E_t \log_2 3 \rceil$ bits, a field spanning $2^{\lceil E_t \log_2 3 \rceil} \geq 3^{E_t}$ values. Hence on a binary fabric a ternary exponent never spans more values per bit than a binary one, with equality only at $E_t = 0$. Writing $\rho(E_t) = 3^{E_t}/2^{\lceil E_t \log_2 3 \rceil}$ for the utilisation, $\rho \in (1/2, 1]$ and ρ does not converge.*

Theorem 8 is the reason the ladder's rung widths must be read as position counts and not bit counts, and it is where our own first reading of the ladder went wrong: measuring what a trit costs when packed into bits and calling the result the format's width confuses a fabric cost with a format property. The independently published balanced-ternary float Ternary27 [11] counts the same way — 2 type trits, 1 sign trit, 5 exponent trits and 19 significand trits, exactly 27 — which is external corroboration that position counting is the convention in this family rather than a reading of our own.

Theorem 9 (Unallocated positions are a pure loss)**.** *Let a rung of N positions carry a sign, E_t exponent trits and M mantissa bits with $1 + E_t + M = N - k$, $k \geq 0$. Then relative to the rung with the same E_t and $M + k$, its expected relative error is larger by exactly 2^k and its dynamic range is identical.*

Proof. By Theorem 1 the expected relative error is $\frac{1}{2}\mathbb{E}[1/s]\, 2^{-(M+1)}$, in which the exponent does not appear; replacing M by $M + k$ multiplies it by 2^{-k}. The dynamic range is 3^{E_t} binades, a function of E_t alone, so it is unchanged. Nothing else in the layout depends on M. □ □

Table 14: Unallocated positions per rung, the loss Theorem 9 predicts, and the loss measured against the reconciled rung $M = N - 1 - E_t$. Exact-arithmetic workload, 800 probes per rung.

rung	k	predicted	measured
GF-T16	2	4×	4.03×
GF-T64	4	16×	15.43×
GF-T128	4	16×	15.99×
GF-T256	4	16×	15.24×

Theorem 9 converts the slack of Section 9 into a number without any measurement, and the measurement then confirms it. Against a workload built in exact arithmetic — necessary above GF-T32, where a double-precision probe measures its own resolution rather than the format's — the predicted and observed factors are:

The final question is one the ladder answers implicitly and, as far as we can find, no one has stated: GF-T encodes its exponent in radix 3 but scales by 2^e, whereas Ternary27 scales by 3^e. These are different choices and they are separable.

Theorem 10 (The scale radix, and why 2 beats 3)**.** *Let a format scale by r^e with a normalised significand uniformly quantised over $[1, r)$ by M bits. Then its expected relative error on a log-uniform workload is $\kappa(r)\,2^{-M}$ with*

$$\kappa(r) \;=\; \frac{(r-1)^2}{r \ln r}, \qquad \kappa(2) = 0.72135, \quad \kappa(3) = 1.21365.$$

Holding the field layout fixed, moving the scale from $r = 2$ to $r = 3$ therefore multiplies the error by $\kappa(3)/\kappa(2) = 1.6825$ while multiplying the dynamic range by $\log_2 3 = 1.5850$. Measured in positions, it costs $\log_2 1.6825 = 0.7506$ of precision and buys $\log_3 1.5850 = 0.4192$ of range: a net loss of 0.3314 positions per number. A power-of-two scale strictly dominates a power-of-three scale at equal width.

Proof. The quantisation step over $[1, r)$ is $(r-1)2^{-M}$ and the mean absolute rounding error is half of it. On a log-uniform workload the significand has density $1/(s \ln r)$ on $[1, r)$, so $\mathbb{E}[1/s] = \frac{1}{\ln r}\int_1^r s^{-2}\,ds = \frac{1-1/r}{\ln r}$. Multiplying gives $\frac{1}{2}(r-1)2^{-M} \cdot \frac{1-1/r}{\ln r}$, and $\frac{1}{2}(r-1)(1-1/r) = (r-1)^2/2r$. Absorbing the factor $\frac{1}{2}$ into the $2^{-(M+1)}$ convention of Theorem 1 yields κ as stated. For the range, E_t trits span 3^{E_t} steps of $\log_2 r$ binades each. One mantissa bit halves the error and one exponent trit triples the range, which fixes the exchange rate between the two currencies and gives the position figures. □ □

Direct measurement at two rungs, encoding and decoding through both scales in exact arithmetic, gives error ratios of 1.675 at $(E_t, M) = (4, 11)$ and 1.690 at $(6, 25)$ against the predicted 1.6825, and range ratios of 1.583 and 1.585 against the predicted 1.5850.

Corollary 6 (What GF-T is, precisely)**.** *GF-T is not a ternary float in the sense of Ternary27; it is a binary-radix float whose exponent is* encoded *in balanced ternary. Theorem 10 says that this is the better of the two available choices by 0.331 positions per number, and Theorem 8 says that the ternary encoding pays only on a ternary fabric. The two together delimit the format exactly: the radix choice is right on any fabric, and the encoding choice is right on one.*

11 The regime field is a universal code

Theorem 6 identifies the exponent's encoding as a prefix code. That identification is worth making explicit, because it places the taxonomy of Section 3.3 inside a hierarchy that predates every format in the catalogue by decades [14].

Table 15: M_{eff} by $|e|$ and usable range for three regime codes at $N = 16$, each normalised to satisfy Kraft. The square-root code is a genuine Pareto point: more precision than the logarithmic code throughout the core, more range than the unary one.

regime	$\lvert e\rvert$=1	4	16	64	256	binades
unary, $\ell \propto \lvert e\rvert$ (posit class)	11	10	7	−5	−53	43
$\ell = \lceil\sqrt{\lvert e\rvert}\rceil$	11	10	8	4	−4	121
$\ell \propto \log_2 \lvert e\rvert$ (takum class)	9	7	5	3	1	511

Theorem 11 (The staircase is the codeword-length function)**.** *Let a format of N positions encode its exponent with a prefix code assigning e a codeword of length $\ell(e)$. Then $M_{\text{eff}}(e) = N - 1 - \ell(e)$, and the staircase form is the growth rate of ℓ and nothing else.*

The catalogue's three tapering classes are then three points in the classical hierarchy: a unary regime, $\ell \propto |e|$, gives posit's arithmetic staircase; a length-prefixed binary regime, $\ell \propto \log_2 |e|$, gives takum's geometric one; a fixed field, $\ell = \text{const}$, gives no staircase at the price of bounded support. Elias' γ, δ and ω codes occupy the logarithmic class at successively finer asymptotics.

Theorem 12 (Logarithmic is the floor)**.** *For any prefix code on the integers, $\ell(e) \geq \log_2 |e|$ for infinitely many e. Hence no format of unbounded range tapers more gently than one bit per doubling of $|e|$, asymptotically.*

Proof. Kraft's inequality gives $\sum_e 2^{-\ell(e)} \leq 1$. If $\ell(e) < \log_2 |e|$ for all but finitely many e, the tail of the sum exceeds $\sum_e 1/|e|$, which diverges. □ □

Theorem 12 is worth stating plainly because it bounds this work as much as its competitors: takum's regime is asymptotically optimal, and no format — GF-T included — can beat it over unbounded range. What GF-T does instead is decline the unbounded range, which Theorem 6 shows is the only other option. The measured advantages of Section 4 are therefore advantages *inside GF-T's range*, which is where the taper has already cost takum the bits, and they should never be read as a claim beyond it.

Theorem 13 (The regime's radix does not matter)**.** *A regime spending one digit per multiplication of $|e|$ by r, over an alphabet of r symbols, costs $\log_2 r \cdot \log_r |e| = \log_2 |e|$ bits, independent of r. The staircase form is invariant to the regime's radix.*

Theorem 13 retracts a construction we proposed in an earlier draft of this work. Having observed the gap between the arithmetic and geometric classes, we suggested a regime spending one trit per tripling of $|e|$ as the natural way for a ternary format to occupy it. It does not occupy it: building both codes and normalising each to satisfy Kraft, the two codeword-length functions differ by an additive constant of exactly 1 at every $|e|$ from 1 to 4096. One trit per tripling is takum's staircase with a different constant, not a third form. A ternary fabric changes what the regime costs to decode; it does not change what it costs to encode.

The gap is real, occupiable, and empty for a reason we can only conjecture. Growth rates strictly between $\log_2 |e|$ and $|e|$ do exist and are Kraft-feasible — $\ell(e) = \lceil\sqrt{|e|}\rceil$ is one — and such a code is not dominated. At $N = 16$, normalised so that all three are legal prefix codes:

For reference, a fixed field reaching the same 121 binades as the square-root code spends 8 bits on the exponent and holds 7 bits flat everywhere — better than the square-root code beyond $|e| \approx 30$ and worse below it, which is the taper trade in miniature.

So the gap is not empty because its occupants are dominated; they are not. Our first explanation was decode cost, and building the hardware refutes it.

Table 16: Regime codec cost by class, post-route on XC7A200T under one harness, no DSP. Figures are net of the harness, which costs 24 LUTs and was synthesised separately and subtracted; the round-trip column is decode plus encode.

class	decode		encode		round trip
	LUTs	$F_{\max}$	LUTs	$F_{\max}$	LUTs
unary, $\ell \propto \lvert e\rvert$ (posit)	379	36.8 MHz	59	211.2 MHz	438
$\ell \propto \log_2 \lvert e\rvert$ (takum)	**6**	**805.8** MHz	**34**	**285.1** MHz	**40**
$\ell = \lceil \sqrt{\lvert e\rvert} \rceil$	393	33.1 MHz	123	96.4 MHz	516
fixed field (GF, GF-T)	0	—	0	—	**0**

11.1 What a regime costs in silicon

We wrote three regime codecs to one interface — a 16-bit word in, an exponent and a mantissa shift out, and the mirror encoder — and synthesised each under a single harness on XC7A200T with Yosys and nextpnr-xilinx, hard multipliers disabled. These are structural models of the three classes, not reproductions of the published posit or takum codecs; what they compare is a run-scanned regime against a length-prefixed one, which is the structural difference the taxonomy names.

The conjecture said the square-root code would be too expensive to be worth building. It is not: it costs 18% more than the unary regime round trip, and the unary regime ships in production. What the measurement does show is that the expense lies somewhere else than we assumed, and this has a closed form.

Theorem 14 (Decode cost is set by the scan, not by the staircase)**.** *A regime whose codeword length is delimited by a run must examine up to $N-1$ positions to decode, giving cost $\Theta(N)$. A regime whose length is carried in a fixed field of b bits requires no scan and a shift of at most 2^b-1, giving cost $\Theta(1)$ in N. The two costs are independent of the growth rate of ℓ, and therefore of the staircase form.*

Theorem 14 predicts the table exactly. The unary and square-root codes share the same 15-position scan and cost 403 and 417 LUTs, a difference of 3.5% that is the squarer and nothing else; the length-prefixed code has no scan and costs 30. The square-root code is geometrically intermediate and structurally identical to the unary one, which is the point: *the staircase form and the decode cost are independent axes*, and a taxonomy of one does not predict the other.

Corollary 7 (The logarithmic regime is not a trade)**.** *Against the unary regime the length-prefixed one is better on every axis measured: asymptotically optimal by Theorem 12,* $11\times$ *smaller round trip, and* $22\times$ *faster to decode. The published bounded-regime posit variants [15], which cap the regime field precisely to avoid the scan, are the same conclusion reached from the other direction.*

Corollary 8 (The measured price of unbounded range)**.** *A fixed field has no regime codec at all. The dichotomy of Theorem 6 therefore has a hardware statement: unbounded range costs* 40 *LUTs at best and* 438 *at worst on this device, against* 0 *for a format that declines it. Section 12 converts those LUTs into the same currency as the accuracy results, which is the only way to say whether they matter.*

We are left without an explanation for the empty gap. It is not domination, since the square-root code is Pareto-undominated; and it is not cost, since it is within 16% of a regime that ships. Both of our explanations are now measured and both are wrong, and we record that rather than replace it with a third.

Table 17: Regime area converted into mantissa bits through Theorem 15. The unary regime costs, in silicon alone, more than GF-T16's entire mantissa.

regime	LUTs	bits at 16-bit class	bits at 32-bit class
unary (posit class)	438	**8.98**	3.94
length-prefixed (takum class)	40	0.82	0.36

12 Converting silicon into precision

Corollary 8 gives a regime's cost in LUTs and Section 4 gives an accuracy ratio, and the two cannot be compared until they are in one currency. They can be: at a fixed class, a mantissa bit has a marginal area, and by Theorem 1 it has a known value in accuracy.

Theorem 15 (Area and precision are commensurable)**.** *Let $\lambda = \mathrm{d}A/\mathrm{d}M$ be the marginal area of a mantissa bit at a given class. A regime codec costing C LUTs is then equivalent to C/λ mantissa bits of silicon, and by Theorem 1 to a factor $2^{C/\lambda}$ in accuracy at equal area.*

Measuring λ directly — the same width-derived multiplier synthesised at $M = 7, 9, \ldots, 25$ under one harness — gives 285, 372, 443, 567, 696, 870, 1238 and 1683 LUTs. The marginal cost is not constant: it rises from 35.5 LUTs per bit between M=9 and M=11 to 111.2 between M=21 and M=25. Taking the local value at each class, $\lambda = 48.8$ at the 16-bit class and 111.2 at the 32-bit one:

12.1 How area grows along the ladder

Theorem 15 needs λ, and λ is a derivative of an area law we can state. Fitting $A(M) = cM^{\alpha}$ to the eight rungs from M=7 to M=25 gives $\alpha = 1.406$ at $R^2 = 0.984$, and combined with Theorem 1 it predicts a scaling relation:

Proposition 16 (Error decays as a stretched exponential in area)**.** *With $\varepsilon \propto 2^{-M}$ and $A \propto M^{\alpha}$, $\ln(1/\varepsilon) \propto A^{1/\alpha}$.*

Measured over $M \in [7, 25]$ — a sixfold range of area and a 250,000-fold range of error — the constant of proportionality holds to within 27% and drifts monotonically downward, so the relation is an approximation and we label it a proposition rather than a theorem. The drift has a cause, and finding it falsified a prediction we registered before synthesising.

The exponent is not constant, and the fit does not extrapolate. We predicted from the $[7, 25]$ fit that the reconciled GF-T64, at $M = 56$, would cost 4,762 LUTs. It costs 7,479 at 48.20 MHz — the prediction is low by 1.57×. Reading α locally rather than globally shows why: it rises monotonically, from 1.06 between M=7 and M=9 to 1.78 between M=15 and M=17 and 1.85 between M=25 and M=56.

Theorem 17 (The ladder's area law)**.** *A rung's area is a fixed overhead — registers, the exponent adder, the renormalisation — plus a significand multiplier quadratic in M:*

$$A(M) \;=\; f + cM^2.$$

Consequently the effective power-law exponent $\alpha(M) = \mathrm{d}\ln A/\mathrm{d}\ln M$ is not a constant but rises monotonically toward 2, and the marginal cost of a mantissa bit, $\lambda = \mathrm{d}A/\mathrm{d}M = 2cM$, is linear *in M rather than fixed.*

Table 18: The same regime codecs valued at five mantissa widths through Theorem 18, with $c = 2.4455$ measured on XC7A200T. The unary regime's silicon exceeds an entire GF-T16 mantissa; the length-prefixed regime is below one bit everywhere above the 8-bit class.

M	λ (LUT/bit)	unary, 438 LUT	length-prefixed, 40 LUT
9	44.0	**9.95**	0.91
11	53.8	8.14	0.74
25	122.3	3.58	0.33
56	273.9	1.60	0.15
90	440.2	1.00	0.09

Synthesising the same width-derived multiplier at fourteen mantissa widths from $M{=}7$ to $M{=}90$ — 285, 372, 443, 567, 696, 870, 1238, 1683, 2601, 3990, 5934, 7479, 12232 and 19980 LUTs — and fitting both parameters gives $f = 141$ LUTs and $c = 2.4455$ LUTs per bit squared at $R^2 = 0.99963$, with a maximum deviation of 9.9% across a seventyfold range of area. Read locally, α climbs from 1.06 between $M{=}7$ and $M{=}9$ to 2.20 between $M{=}56$ and $M{=}70$, which is Theorem 17 rather than noise. On the registered prediction the structural model is right to +4.4% where the single power law was low by 36.3%; we report both, since the power law was ours and we published the prediction before synthesising.

Theorem 18 (A regime's worth in precision falls as $1/M$)**.** *By Theorems 15 and 17, a regime codec of fixed area C is worth*

$$\frac{C}{\lambda} = \frac{C}{2cM}$$

mantissa bits, inversely proportional to the mantissa width. It is worth at least one mantissa bit exactly while $M \leq C/2c$.

Corollary 9 (Break-even widths)**.** *With $c = 2.4455$, the unary regime is worth at least one mantissa bit of silicon up to $M = 89.6$ — that is, through the entire 128-bit class — while the length-prefixed regime reaches one bit only up to $M = 8.2$. takum's regime therefore costs less than a single mantissa bit of silicon at every rung above GF-T8, and posit's costs more than one at every rung below GF-T256.*

Corollary 9 is the exact form of what Corollary 11 could only say qualitatively, and it is the sharpest statement we can make about where a fixed field pays for itself in area: against posit, almost everywhere; against takum, only at the very bottom of the ladder.

Corollary 10 (Range is paid for three times, and they are commensurable)**.** *A tapered format pays for unbounded range in three currencies: word positions, in the regime field; silicon, in the codec; and precision at the extremes, in the taper. Theorem 15 and Theorem 1 put all three in mantissa bits, so they can be added.*

Applied honestly, Corollary 10 separates the two tapered families rather than lumping them. Against posit the silicon term alone is 8.98 bits at the 16-bit class — more than GF-T16's entire 9-bit mantissa, so the area a posit implementation spends decoding its regime would, spent on significand instead, roughly double the precision budget of a fixed-field format at the same class. Against takum the silicon term is 0.82 bits, which is small. The length-prefixed regime is cheap, and any argument for a fixed field made on area grounds is an argument against posit and not against takum. Against takum what remains is the accuracy result of Section 4, bounded as Theorem 12 requires to GF-T's own range.

Corollary 11 (The area argument is a narrow-width argument)**.** *Since λ grows superlinearly in M while a regime codec's cost is roughly fixed, the same codec converts to fewer mantissa bits*

Table 19: Published 32-bit decoders, post-route on XC7A200T under one harness, no DSP, converting to binary32. These are the real codecs rather than the structural models of Table 16.

decoder	LUTs	$F_{\max}$	what dominates
`posit32_decode`	480	49.0 MHz	leading-zero count, barrel shift
`takum32_decode`	76,821	—	a 65536×48-bit table for 2^f
GF-T32, exponent path	0	—	the field is read directly

as the class widens: 8.98 *bits at 16 becomes* 3.94 *at 32 for the unary regime. The fixed-field area advantage is therefore strongest where the words are narrowest, which is the regime in which quantised inference operates and not the one in which numerical linear algebra does.*

13 The value law costs more than the field layout

Sections 11.1 and 12 priced the regime as a field to be extracted, and synthesising the published decoders shows that this is the smaller half of the question. We take the takum and posit decoders from an existing open verification suite — Verilog, golden-checked against an mpmath oracle — and put them through the same harness on XC7A200T.

The takum figure needs stating carefully. Its decoder evaluates $2^{f_{\mathrm{hi}}/2^{16}}$ from a 65536×48-bit table, then applies a Taylor correction through a 108-bit and an 80-bit multiply. In this flow the table did not infer as block RAM — the module lacks the `ram_style="block"` attribute that the same suite's 16-bit decoder carries — so it was built from logic, and 76,821 LUTs is what the design as written costs here rather than a property of takum. Inferred correctly the table is 3.15 Mbit, which is 85 RAMB36 tiles, 23% of this device's entire block memory. Either way the cost is not the regime field.

Theorem 19 (Linear and logarithmic value laws)**.** *Let a format's value law be $v = s \cdot 2^e$ with s and e read from fields (*linear*), or $v = \pm 2^{\ell}$ for a decoded real ℓ (*logarithmic*). A linear law decodes with a shift, cost independent of the mantissa width. A logarithmic law requires evaluating an exponential, whose cost is a table or an iterative kernel and is independent of the field layout — therefore independent of the staircase form of Section 3.3.*

Theorem 19 is a second axis, orthogonal to the first. The staircase taxonomy classifies how a format spends *precision*; the value law classifies what it costs to *read*. posit is tapered and linear; takum is tapered and logarithmic; LNS is untapered and logarithmic; GF, GF-T and IEEE are untapered and linear. The two axes are independent, and our earlier reading — that decode cost is set by whether the regime is scanned (Theorem 14) — describes only the field-extraction term. It remains correct within that term, and Table 19 shows the term is not where the money is for a logarithmic format.

Corollary 12 (What GF-T's exponent path costs)**.** *GF-T's value law is linear and its exponent field is fixed, so its exponent path is wires: no scan, no table, no iteration. Against posit32 that is* 480 *LUTs not spent; against takum32 it is a table of* 3.15 *Mbit not instantiated. This is a larger and more robust statement than the accuracy result, and unlike it, Theorem 12 places no range bound on it.*

We flag one asymmetry rather than let it pass. A logarithmic value law buys something real in exchange: multiplication becomes addition. The comparison above is of *decoders* — the cost of getting a value into a binary datapath — and a design that stays in the logarithmic domain never pays it. That is the regime in which LNS accelerators are built, and against those the argument of Corollary 12 does not apply at all.

14 Limitations

1. **The comparison is against takum16, not tekum16.** The oracle labelled tekum decodes all 65,536 16-bit codes identically to the takum oracle. We report the comparison as against takum and note that a genuine tekum comparison remains to be done.

2. **The reference oracle carried a sign defect, now fixed.** Round-tripping 1.5 through the parameterised oracle returned −1.5 at mantissa widths of 21 and 25 bits: the sign bit sat at a fixed position while the exponent mask took more bits than the field holds, so the two overlapped. Both are now derived from the format parameters, and GF-T32's round-trip error falls from 5.4e−1 — what a systematically inverted sign averages to — to 5.3e−9.

   We record how it survived, because the lesson outlives the bug. The ladder's check was add/mul commutativity, and an inverted sign survives on both sides of $a + b = b + a$. The check was real and blind to the class. A round-trip assertion sees it, costs one line per probe, and now runs at all nine rungs. tekum [2] is a descendant of takum [1] adapted for balanced ternary. Our oracle reconstructs it from takum's field scheme; the full per-trit specification requires the tekum paper. The ratios are as good as that model.

3. **No head-to-head in hardware.** takum's codec RTL is public and written in VHDL [3]. We did not synthesise it alongside GF-T, so the cost figures are GF-T's own. What is visible in that source, and which we report as structure rather than measurement: the 16-bit codec instantiates a 725-line generated leading-zero-counter and barrel shifter — the regime decode a fixed-field format does not have.

4. **The range is bounded** at ±40 in powers of two, roughly ±12 decades, where a regime field is unbounded. Fixed fields buy the cheap datapath and the uniform mantissa; range is the price, and workloads that need more should raise the trit budget or use a tapered format.

5. **Accuracy above GF-T32 is limited by the measuring instrument.** Once the oracle defect was fixed, GF-T32 measures 5.3e−9, flat across the workload, which is what 25 mantissa bits should give. GF-T64 carries 52 mantissa bits, which is exactly the significand of the IEEE double our metric is computed in, so the figure it returns describes the metric rather than the format. Measuring the upper rungs requires exact rationals throughout, as the oracle itself already uses.

6. **Four of nine rungs are measured in hardware.** GF-T4 through GF-T32 fit the device; GF-T64 and GF-T128 are within reach of a larger part and GF-T256 upwards exceed a single Artix-7 fabric entirely. Table 8 is the specification and the oracle's coverage; Table 7 is what was placed and routed. We keep them apart on purpose.

7. **One device family.** Artix-7 on an open flow. Not multi-corner characterisation; an ASIC mapping will differ.

8. **The ternary-fabric argument is architectural.** No ternary process exists to synthesise on. Section 6 is a binary-fabric result; the ternary claim is reasoning about regime decode and native exponent addition and should be read as such.

Reproducibility

The reference oracle, the RTL, the equivalence testbenches and the synthesis commands are public. The toolchain is Yosys 0.65, nextpnr-xilinx 1743d0f, Icarus Verilog 13.0 and Python 3.14 — all open-source, none requiring a licence, which is the point: a result nobody can reproduce without buying something is not a public result.

Disclosure

An AI coding assistant was used in preparing the software, the measurements' tooling and this manuscript. Every figure reported here was produced by running the named tools on hardware in the author's possession; no measurement in this paper is estimated, and where a figure is derived rather than observed it is labelled.

References

[1] L. Hunhold. Takum arithmetic. arXiv:2404.18603.

[2] L. Hunhold. Tekum: balanced-ternary tapered-precision arithmetic. arXiv:2512.10964.

[3] L. Hunhold. Design and implementation of a takum arithmetic hardware codec in VHDL. arXiv:2408.10594. Source: https://github.com/takum-arithmetic/Takum-Codec-RTL.

[4] D. Vasilev. GoldenFloat: a φ-derived static-split floating-point family from GF4 to GF1024 with a Lucas-exact integer identity. arXiv:2606.05017v3, 2026.

[5] D. Vasilev. An 83-format numeric catalog with bit-exact conformance vectors: a vendor-neutral reference for FP8, BF16, MXFP4, and microscaling formats. arXiv:2606.09686v2, 2026.

[6] J. L. Gustafson and I. Yonemoto. Beating floating point at its own game: posit arithmetic. *Supercomputing Frontiers and Innovations*, 4(2), 2017. Posit Standard (2022) fixes $es = 2$ at all precisions.

[7] C. Wolf. Yosys open synthesis suite. https://yosyshq.net/yosys/.

[8] D. Shah et al. nextpnr: a portable FPGA place-and-route tool. https://github.com/YosysHQ/nextpnr; Xilinx target via Project X-Ray.

[9] B. Hayes, "Third Base," *American Scientist*, vol. 89, no. 6, pp. 490–494, 2001.

[10] Radix economy, optimal radix choice. https://en.wikipedia.org/wiki/Radix_economy

[11] Ternary27: A Balanced Ternary Floating Point Format, Version 3.1. https://hackaday.io/project/183153

[12] D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[13] IBM hexadecimal floating-point. https://en.wikipedia.org/wiki/IBM_hexadecimal_floating-point

[14] P. Elias, "Universal codeword sets and representations of the integers," *IEEE Trans. Inform. Theory*, vol. 21, no. 2, pp. 194–203, 1975.

[15] Closing the Gap Between Float and Posit Hardware Efficiency, arXiv:2603.01615.

[16] Performance-Efficiency Trade-off of Low-Precision Numerical Formats in Deep Neural Networks, arXiv:1903.10584.